Calculus

This section covers how to do basic calculus tasks such as derivatives, integrals, limits, and series expansions in SymPy. If you are not familiar with the math of any part of this section, you may safely skip it.

```
>>> from sympy import *
>>> x, y, z = symbols('x y z')
>>> init_printing(use_unicode=True)
```

Derivatives

To take derivatives, use the `diff` function.

```
>>> diff(cos(x), x)
-sin(x)
>>> diff(exp(x**2), x)
     ⎛ 2⎞
     ⎝x ⎠
2⋅x⋅ℯ
```

`diff` can take multiple derivatives at once. To take multiple derivatives, pass the variable as many times as you wish to differentiate, or pass a number after the variable. For example, both of the following find the third derivative of x^4.

```
>>> diff(x**4, x, x, x)
24⋅x
>>> diff(x**4, x, 3)
24⋅x
```

You can also take derivatives with respect to many variables at once. Just pass each derivative in order, using the same syntax as for single variable derivatives. For example, each of the following will compute $\frac{\partial^7}{\partial x \partial y^2 \partial z^4} e^{xyz}$.

```
>>> expr = exp(x*y*z)
>>> diff(expr, x, y, y, z, z, z, z)
 3  2 ⎛ 3  3  3       2  2  2                 ⎞  x⋅y⋅z
x ⋅y ⋅⎝x ⋅y ⋅z  + 14⋅x ⋅y ⋅z  + 52⋅x⋅y⋅z + 48⎠⋅ℯ
>>> diff(expr, x, y, 2, z, 4)
 3  2 ⎛ 3  3  3       2  2  2                 ⎞  x⋅y⋅z
x ⋅y ⋅⎝x ⋅y ⋅z  + 14⋅x ⋅y ⋅z  + 52⋅x⋅y⋅z + 48⎠⋅ℯ
>>> diff(expr, x, y, y, z, 4)
 3  2 ⎛ 3  3  3       2  2  2                 ⎞  x⋅y⋅z
x ⋅y ⋅⎝x ⋅y ⋅z  + 14⋅x ⋅y ⋅z  + 52⋅x⋅y⋅z + 48⎠⋅ℯ
```

Skip to content

`diff` can also be called as a method. The two ways of calling `diff` are exactly the same, and are provided only for convenience.

```
>>> expr.diff(x, y, y, z, 4)
 3  2 ⎛ 3  3  3       2  2  2                ⎞  x·y·z
x ·y ·⎝x ·y ·z  + 14·x ·y ·z  + 52·x·y·z + 48⎠·ℯ
```

To create an unevaluated derivative, use the `Derivative` class. It has the same syntax as `diff`.

```
>>> deriv = Derivative(expr, x, y, y, z, 4)
>>> deriv
     7
    ∂      ⎛ x·y·z⎞
──────────(ℯ      )
  4   2
∂z  ∂y  ∂x
```

To evaluate an unevaluated derivative, use the `doit` method.

```
>>> deriv.doit()
 3  2 ⎛ 3  3  3       2  2  2                ⎞  x·y·z
x ·y ·⎝x ·y ·z  + 14·x ·y ·z  + 52·x·y·z + 48⎠·ℯ
```

These unevaluated objects are useful for delaying the evaluation of the derivative, or for printing purposes. They are also used when SymPy does not know how to compute the derivative of an expression (for example, if it contains an undefined function, which are described in the Solving Differential Equations section).

Derivatives of unspecified order can be created using tuple `(x, n)` where `n` is the order of the derivative with respect to `x`.

```
>>> m, n, a, b = symbols('m n a b')
>>> expr = (a*x + b)**m
>>> expr.diff((x, n))
  n
 ∂ ⎛          m⎞
──⎝(a·x + b) ⎠
  n
∂x
```

Integrals

To compute an integral, use the `integrate` function. There are two kinds of integrals, definite and indefinite. To compute an indefinite integral, that is, an antiderivative, or primitive, just pass the variable after the expression.

Skip to content

```
(cos(x), x)
sin(x)
```

Note that SymPy does not include the constant of integration. If you want it, you can add one yourself, or rephrase your problem as a differential equation and use `dsolve` to solve it, which does add the constant (see Solving Differential Equations).

Quick Tip

∞ in SymPy is `oo` (that's the lowercase letter "oh" twice). This is because `oo` looks like ∞, and is easy to type.

To compute a definite integral, pass the argument `(integration_variable, lower_limit, upper_limit)`. For example, to compute

$$\int_0^\infty e^{-x}\,dx,$$

we would do

```
>>> integrate(exp(-x), (x, 0, oo))
1
```

As with indefinite integrals, you can pass multiple limit tuples to perform a multiple integral. For example, to compute

$$\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} e^{-x^2-y^2}\,dx\,dy,$$

do

```
>>> integrate(exp(-x**2 - y**2), (x, -oo, oo), (y, -oo, oo))
π
```

If `integrate` is unable to compute an integral, it returns an unevaluated `Integral` object.

```
>>> expr = integrate(x**x, x)
>>> print(expr)
Integral(x**x, x)
>>> expr
⌠
⎮  x
⎮ x  dx
⌡
```

Skip to content

`ve`, you can create an unevaluated integral using `Integral`. To later evaluate `loit`.

```
>>> expr = Integral(log(x)**2, x)
>>> expr
⌠
⎮    2
⎮ log (x) dx
⌡
>>> expr.doit()
         2
x·log (x) - 2·x·log(x) + 2·x
```

`integrate` uses powerful algorithms that are always improving to compute both definite and indefinite integrals, including heuristic pattern matching type algorithms, a partial implementation of the Risch algorithm, and an algorithm using Meijer G-functions that is useful for computing integrals in terms of special functions, especially definite integrals. Here is a sampling of some of the power of `integrate`.

```
>>> integ = Integral((x**4 + x**2*exp(x) - x**2 - 2*x*exp(x) - 2*x -
...     exp(x))*exp(x)/((x - 1)**2*(x + 1)**2*(exp(x) + 1)), x)
>>> integ
⌠
⎮ ⎛ 4    2  x    2        x          x⎞  x
⎮ ⎝x  + x ·ℯ  - x  - 2·x·ℯ  - 2·x - ℯ ⎠·ℯ
⎮ ──────────────────────────────────────── dx
⎮                2        2 ⎛ x    ⎞
⎮       (x - 1) ·(x + 1) ·⎝ℯ  + 1⎠
⌡
>>> integ.doit()
                  x
   ⎛ x    ⎞     ℯ
log⎝ℯ  + 1⎠ + ──────
               2
              x  - 1
```

```
>>> integ = Integral(sin(x**2), x)
>>> integ
⌠
⎮    ⎛ 2⎞
⎮ sin⎝x ⎠ dx
⌡
>>> integ.doit()
                ⎛√2·x⎞
3·√2·√π·S⎜──────⎟·Γ(3/4)
         ⎝ √π   ⎠
──────────────────────
        8·Γ(7/4)
```

Skip to content

```
>>> integ = Integral(x**y*exp(-x), (x, 0, oo))
>>> integ
∞
⌠
⎮  y  -x
⎮ x ⋅ℯ   dx
⌡
0
>>> integ.doit()
⎧ Γ(y + 1)    for re(y) > -1
⎪
⎪∞
⎪⌠
⎨⎮  y  -x
⎪⎮ x ⋅ℯ   dx    otherwise
⎪⌡
⎪0
⎩
```

This last example returned a `Piecewise` expression because the integral does not converge unless $\Re(y) > -1$.

Numeric Integration

Numeric integration is a method employed in mathematical analysis to estimate the definite integral of a function across a simplified range. SymPy not only facilitates symbolic integration but also provides support for numeric integration. It leverages the precision capabilities of the `mpmath` library to enhance the accuracy of numeric integration calculations.

```
>>> from sympy import Integral, Symbol, sqrt
>>> x = Symbol('x')
>>> integral = Integral(sqrt(2)*x, (x, 0, 1))
>>> integral
1
⌠
⎮ √2⋅x dx
⌡
0
>>> integral.evalf()
0.707106781186548
```

To compute the integral with a specified precision:

```
>>> integral.evalf(50)
0.70710678118654752440084436210484903928483593768847
```

Numeric integration becomes a viable approach in situations where symbolic integration is impractical or impossible. This method allows for the computation of integrals through numerical

Skip to content

when dealing with infinite intervals or integrands:

```
>>> Integral(exp(-(x ** 2)), (x, -oo, oo)).evalf()
1.77245385090552
```

```
>>> Integral(1 / sqrt(x), (x, 0, 1)).evalf()
2.00000000000000
```

Limits

SymPy can compute symbolic limits with the `limit` function. The syntax to compute

$$\lim_{x \to x_0} f(x)$$

is `limit(f(x), x, x0)`.

```
>>> limit(sin(x)/x, x, 0)
1
```

`limit` should be used instead of `subs` whenever the point of evaluation is a singularity. Even though SymPy has objects to represent ∞, using them for evaluation is not reliable because they do not keep track of things like rate of growth. Also, things like $\infty - \infty$ and $\frac{\infty}{\infty}$ return nan (not-a-number). For example

```
>>> expr = x**2/exp(x)
>>> expr.subs(x, oo)
nan
>>> limit(expr, x, oo)
0
```

Like `Derivative` and `Integral`, `limit` has an unevaluated counterpart, `Limit`. To evaluate it, use `doit`.

```
>>> expr = Limit((cos(x) - 1)/x, x, 0)
>>> expr
     ⎛cos(x) - 1⎞
 lim ⎜──────────⎟
x─→0⁺⎝    x     ⎠
>>> expr.doit()
0
```

To evaluate a limit at one side only, pass `'+'` or `'-'` as a fourth argument to `limit`. For example, to compute

Skip to content

$$\lim_{x \to 0^+} \frac{1}{x},$$

do

```
>>> limit(1/x, x, 0, '+')
∞
```

As opposed to

```
>>> limit(1/x, x, 0, '-')
-∞
```

Series Expansion

SymPy can compute asymptotic series expansions of functions around a point. To compute the expansion of $f(x)$ around the point $x = x_0$ terms of order x^n, use `f(x).series(x, x0, n)`. `x0` and `n` can be omitted, in which case the defaults `x0=0` and `n=6` will be used.

```
>>> expr = exp(sin(x))
>>> expr.series(x, 0, 4)
         2
        x     ⎛ 4⎞
1 + x + ── + O⎝x ⎠
        2
```

The $O\left(x^4\right)$ term at the end represents the Landau order term at $x = 0$ (not to be confused with big O notation used in computer science, which generally represents the Landau order term at x where $x \to \infty$). It means that all x terms with power greater than or equal to x^4 are omitted. Order terms can be created and manipulated outside of `series`. They automatically absorb higher order terms.

```
>>> x + x**3 + x**6 + O(x**4)
     3    ⎛ 4⎞
x + x  + O⎝x ⎠
>>> x*O(1)
O(x)
```

If you do not want the order term, use the `removeO` method.

```
>>> expr.series(x, 0, 4).removeO()
 2
x
── + x + 1
2
```

The `O` notation supports arbitrary limit points (other than 0):

```
>>> exp(x - 6).series(x, x0=6)
                      3          4          5
                (x - 6)    (x - 6)    (x - 6)           ⎛        6        ⎞
-5 + ──────── + ──────── + ──────── + ──────── + x + O⎝(x - 6) ; x → 6⎠
        2          6          24         120
```

Skip to content

Finite differences

So far we have looked at expressions with analytic derivatives and primitive functions respectively. But what if we want to have an expression to estimate a derivative of a curve for which we lack a closed form representation, or for which we don't know the functional values for yet. One approach would be to use a finite difference approach.

The simplest way the differentiate using finite differences is to use the `differentiate_finite` function:

```
>>> f, g = symbols('f g', cls=Function)
>>> differentiate_finite(f(x)*g(x))
-f(x - 1/2)·g(x - 1/2) + f(x + 1/2)·g(x + 1/2)
```

If you already have a `Derivative` instance, you can use the `as_finite_difference` method to generate approximations of the derivative to arbitrary order:

```
>>> f = Function('f')
>>> dfdx = f(x).diff(x)
>>> dfdx.as_finite_difference()
-f(x - 1/2) + f(x + 1/2)
```

here the first order derivative was approximated around x using a minimum number of points (2 for 1st order derivative) evaluated equidistantly using a step-size of 1. We can use arbitrary steps (possibly containing symbolic expressions):

```
>>> f = Function('f')
>>> d2fdx2 = f(x).diff(x, 2)
>>> h = Symbol('h')
>>> d2fdx2.as_finite_difference([-3*h,-h,2*h])
f(-3·h)   f(-h)   2·f(2·h)
─────── - ───── + ────────
     2        2         2
  5·h      3·h     15·h
```

If you are just interested in evaluating the weights, you can do so manually:

```
>>> finite_diff_weights(2, [-3, -1, 2], 0)[-1][-1]
[1/5, -1/3, 2/15]
```

note that we only need the last element in the last sublist returned from `finite_diff_weights`. The reason for this is that the function also generates weights for lower derivatives and using fewer points (see the documentation of `finite_diff_weights` for more details).

If using `finite_diff_weights` directly looks complicated, and the `as_finite_difference` Skip to content `itive` instances is not flexible enough, you can use `apply_finite_diff` which `_list`, `y_list` and `x0` as parameters:

```
>>> x_list = [-3, 1, 2]
>>> y_list = symbols('a b c')
>>> apply_finite_diff(1, x_list, y_list, 0)
  3·a   b   2·c
- ─── - ─ + ───
   20   4    5
```

Copyright © 2024 SymPy Development Team
Made with Sphinx and @pradyunsg's Furo
Last updated on Sep 18, 2024